Invisible Surfing

Liran Ma and Zhipeng Cai
Department of Computer Science
Texas Christian University

Copyright © 2021 Liran Ma, Texas Christian University.
The development of this document is partially funded by the National Science Foundation's and SaTC program under award No. DGE-1912755. and Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.2 or any later version published by the Free Software Foundation. A copy of the license can be found at http://www.gnu.org/licenses/fdl.html.

Contents

1 SSL/TLS VPN (Certificates) **1**

2 IPsec Host-to-Host VPN (Pre-Shared Key) **7**

3 Secure Shell (SSH) Port Forwarding **12**

4 General Questions **16**

1 SSL/TLS VPN (Certificates)

A Secure Socket Layer Virtual Private Network (SSL VPN) lets remote users access Web applications, client-server apps, and internal network utilities and directories without the need for specialized client software. All traffic between a Web browser and SSL VPN device is encrypted with the SSL protocol, or its successor protocol (TLS). An SSL VPN generally provides two things: secure remote access via a web portal, and network-level access via an SSL-secured tunnel between the client and the corporate network.

In this lab task, we will setup a SSL/TLS VNP service using OpenVPN. You will be able to observe the SSL/TLS handshake procedure. Here, the mutual authentication of server and client is done with certificates (instead of passwords).

1.1 Lab Prep

Download the GitHub repository that contains the Docker configuration files for this lab.

```
git pull
# If you have not downloaded the repository previously,
# you will need to use git clone instead of pull
# git clone https://github.com/liranmatcu/Docker_Security_Labs.git
```

Start a VPN server container instance as follows.

```
cd  Docker_Security_Labs/ssl_vpn

# Build the Docker image for the SSL lab
docker build -t ssl .   # Note that there is a dot in the end

# Start a VPN server container
docker run -it --name server --privileged --hostname vpn-server
-v "$PWD"/server:/ssl_data --rm --device=/dev/net/tun ssl bash
# The above two lines are a single command
```

1.2 Build Certificate Authority (CA)

Instructions below are adopted from the following source with modifications: https://www.itsfullofstars.de/2018/04/setup-openvpn-server-on-amazon-ec2/.

On your VPN server container instance:

```
mkdir -p /etc/openvpn/easy-rsa
# If not root, need to add sudo

cd /etc/openvpn/easy-rsa

cp -Rv /usr/share/easy-rsa/*  ./

./easyrsa init-pki

./easyrsa build-ca
```

The final command (**build-ca**) will build the certificate authority (CA) certificate and key by invoking the interactive **openssl** command:

```
Generating a 2048 bit RSA private key
............++++++
```

```
Enter PEM pass phrase:
...........
Common Name (eg, your name or your server's hostname) []:CA4VPN
...........
```

Note that the only parameters that must be explicitly entered are the **Common Name** and the **PEM pass phrase**.

Next, generate Diffie Hellman parameters. Diffie Hellman parameters must be generated for the OpenVPN server.

```
./easyrsa gen-dh
```

1.3 Server Certificate Generation

Generate a certificate request for the VPN server:

```
./easyrsa  gen-req  server
```

When the **Common Name** is prompted, enter "server".

Next, sign the request.

```
./easyrsa sign-req server server
```

You need to enter the CA's pass phrase to sign the request.

Generate *ta.key* for TLS-AUTH (TLS authentication).

```
cd /etc/openvpn
openvpn --genkey --secret ta.key
mv /etc/openvpn/ta.key /etc/openvpn/easy-rsa/
```

1.4 Client Certificate Generation and Transportation

Generating a VPN client certificate is very similar to the server's. Commands are given as follows.

```
cd /etc/openvpn/easy-rsa/
./easyrsa gen-req client
./easyrsa sign-req client client
```

For each client certificate, make sure to type the appropriate **Common Name** when prompted, i.e., type "**client**" in our scenario. Always use a unique common name for each client. The client certificate is created at: */etc/openvpn/easy-rsa/pki/issued/client.crt*.

Next is to transport all the necessary files to the client. To be specific, we need to transport "ca.crt", "client.crt", "client.key", and "ta.key" to the VPN client.

You can use the shared folder (i.e., */ssl_data*) to transport the files. Specifically, copy these files to the */ssl_data* folder on the server container.

```
cp /etc/openvpn/easy-rsa/pki/ca.crt   /ssl_data/
cp /etc/openvpn/easy-rsa/pki/private/client.key  /ssl_data/
cp /etc/openvpn/easy-rsa/pki/issued/client.crt   /ssl_data/
cp /etc/openvpn/easy-rsa/ta.key  /ssl_data/
```

If your Docker container is running on a local machine (e.g., MAC or PC), the */ssl_data* folder is mounted to the *server* folder in Finder (Mac) or File Explorer (Windows). In Finder or File Explorer, copy these files from the *server* folder and paste them to the *client* folder, which will be mounted to the */ssl_data* folder in the client container. As a result, these files will appear in the */ssl_data* folder in the client container.

For AWS container users, you can use the *cp* command to transport the files since both containers are running on the same AWS VM. Example cp commands are given below.

```
sudo cp  full-path-to/ssl_vpn/server/ca.crt   full-path-to/ssl_vpn/client/
...
```

1.5 Server Configuration

The server configuration file (*server.conf*) has been loaded inside your Docker instance. You only need to copy it from the */ssl_data* folder to the */etc/openvpn* folder.

```
cp /ssl_data/server.conf /etc/openvpn
```

Below is a list of key parameters in server.conf for your reference.

```
ca  /etc/openvpn/easy-rsa/pki/ca.crt
cert /etc/openvpn/easy-rsa/pki/issued/server.crt
key /etc/openvpn/easy-rsa/pki/private/server.key
dh /etc/openvpn/easy-rsa/pki/dh.pem  # no 2048
tls-auth /etc/openvpn/easy-rsa/ta.key 0
```

1.6 Start VPN Server

Before starting the VPN service, check the server's IP address as follows:

```
ifconfig  eth0
```

Write down the IP address for future use.

Start the VPN service on the server via:

```
openvpn /etc/openvpn/server.conf
```

During startup, you need to provide the passphrase of the server certificate.

A normal server startup should look like this (output may vary):

```
Diffie-Hellman initialized with 2048 bit key
Enter Private Key Password:
.......
Initialization Sequence Completed
```

1.6.1 Provide a screenshot showing your VPN server startup information.

The VPN server is running and waiting for a VPN client to connect to it. Next, we will configure and start a VPN client.

1.7 Client Configuration

Open another terminal on your local machine, move to your GitHub folder, and issue the following commands to tart a VPN client container instance.

```
cd  Docker_Security_Labs/ssl_vpn
```

```
Start a VPN client container instance:
  docker run -it --name client --privileged --hostname vpn-client
  -v "$PWD"/client:/ssl_data --rm --device=/dev/net/tun ssl bash
  # The above two lines are a single command
```

On your VPN client, copy ‘ca.crt”, “client.crt”, “client.key” and “ta.key” to the */etc/openvpn/* folder on the client container.

```
cp /ssl_data/*.crt  /etc/openvpn/
cp /ssl_data/*.key  /etc/openvpn/
```

The client configuration file (*client.conf*) has also been created. Copy the *client.conf* file from the */ssl_data* folder to the */etc/openvpn* folder.

```
cp /ssl_data/client.conf  /etc/openvpn/
```

The key entries in client.conf are listed below for your reference.

```
ca        /etc/openvpn/ca.crt
cert      /etc/openvpn/client.crt
key       /etc/openvpn/client.key
tls-auth  /etc/openvpn/ta.key 1
```

1.8 Start VPN Client

Open the */etc/hosts* file, and add an entry as follows:

```
vi /etc/hosts

IP-address-of-the-VPN-server   my-server-1
# In my environment, IP-address-of-the-VPN-server is 172.17.0.2, thus
# mine is ``172.17.0.2   my-server-1''
```

You should have used ifconfig command to check the IP address of your VPN server previously. Fill the correct IP address in the file.

Next, start the VPN service on client via:

```
openvpn   /etc/openvpn/client.conf
```

A normal client startup will look similar to the server output above, and should end with the *Initialization Sequence Completed* message.

1.8.1 Provide a screenshot showing your VPN client startup information.

1.8.2 Observe VPN server activities. What steps defined in the SSL/TLS handshake process can you observe? Provide screenshots showing these steps.

Open another terminal to your VPN client, try a ping to the server across the VPN from the client:

```
docker exec -it client bash
ping 10.8.0.1
```

If ping succeeds, you now have a functional SSL based VPN.

Next, start tcpdump on the server to capture the ongoing ping packets.

```
docker exec -it server bash

tcpdump -vv -i tun0
# Show ping in ICMP

tcpdump -vv -i eth0
# Show ping in encrypted format
```

1.8.3 Why ping 10.8.0.1? What is my-server-1? Please refer to the client.conf file.

1.8.4 Why the ping packets are encrypted in the eth0 capture but not in the tun0 capture?

An example tcpdump capture on tun0 is shown in Fig. 1.

```
root@vpn-server: ~ $ tcpdump -vv -i tun0
tcpdump: listening on tun0, link-type RAW (Raw IP), capture size 262144 bytes
22:18:15.236865 IP (tos 0x0, ttl 64, id 31719, offset 0, flags [DF], proto ICMP (1), length 84)
    10.8.0.6 > 10.8.0.1: ICMP echo request, id 22, seq 29, length 64
22:18:15.236914 IP (tos 0x0, ttl 64, id 13011, offset 0, flags [none], proto ICMP (1), length 84)
    10.8.0.1 > 10.8.0.6: ICMP echo reply, id 22, seq 29, length 64
```

Figure 1: Example tcpdump capture on tun0.

An example tcpdump capture on eth0 is shown in Fig. 2.

```
root@vpn-server: ~ $ tcpdump -vv -i eth0
tcpdump: listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
22:08:10.084175 IP (tos 0x0, ttl 64, id 61955, offset 0, flags [DF], proto UDP (17), length 136)
    172.17.0.3.38028 > vpn-server.1194: [bad udp cksum 0x58ad -> 0x9dd2!] UDP, length 108
22:08:10.084294 IP (tos 0x0, ttl 64, id 38308, offset 0, flags [DF], proto UDP (17), length 136)
    vpn-server.1194 > 172.17.0.3.38028: [bad udp cksum 0x58ad -> 0xdf20!] UDP, length 108
```

Figure 2: Example tcpdump capture on eth0.

1.8.5 In your own words, explain what we have achieved in this section?

2 IPsec Host-to-Host VPN (Pre-Shared Key)

IP security (IPsec) can be configured to connect one host to another by way of a host-to-host connection. This type of connection uses the network to which each host is connected to create the secure tunnel to each other. The requirements of a host-to-host connection are minimal, as is the configuration of IPsec on each host. The hosts need only a dedicated connection to a carrier network (such as the Internet) and Linux to create the IPsec connection.

2.1 Lab Prep

Download the GitHub repository that contains the Docker configuration files for this lab.

```
git pull https://github.com/liranmatcu/Docker_Security_Labs.git
# If you have not downloaded the repository previously,
# you will need to use git clone instead of pull
```

Create your IPsec container instances (a client and a server) as follows.

```
cd   Docker_Security_Labs/ipsec/

# If instances named client and server exist,
# we need to remove them first
```

```
docker  rm  -f  client  server

# Build the image for IPsec lab
docker build -t ipsec .   # Note that there is a dot in the end

# Start IPsec client first
docker run -it --privileged --name client --hostname ipsec-client
-v "$PWD"/client:/data --rm ipsec bash
# The above two lines are a single command

# Start IPsec server
docker run -it --privileged --name server --hostname ipsec-server
-v "$PWD"/server:/data --rm ipsec bash
# The above two lines are a single command
```

2.2 Information Gathering

The first step is to gather network information from each container instance. For a host-to-host connection, you need the following information:

- The IP addresses of the IPsec client and server container instances. You can run the **ifconfig** command inside the container instance to check its IP address. You can obtain this information via the **docker inspect** command. Below is the information in my setup.
  - Client container IP is 172.17.0.2
  - Server container IP is 172.17.0.3
  - Typically, you should have the same IP addresses as mine.
- Determine a secret to be used to initiate the connection and exchange encryption keys during the session. In our example configuration file, the secret is realsecure (in the psk.txt file).

In your IPsec server's terminal, ping the client IP address (172.17.0.2). You should be able to see the ping reply continuously.

```
ping  client-IP-address (i.e., 172.17.0.2)
```

Open another terminal into your IPsec server container, monitor the network traffic using tcpdump.

```
docker exec -it server bash
tcpdump  -n  -i  eth0
```

2.3 Server Configuration

Open another terminal into your IPsec server container (so far, three terminals attached the server), set right permissions, and copy IPsec related configuration files (*racoon.conf*, *setkey.conf*, and *psk.txt*) to the */etc* folder.

```
docker exec -it server bash
chmod 600 /data/psk.txt
cp   /data/*   /etc/
```

If your client IP address is 172.17.0.2 and server IP address is 172.17.0.3, you do **NOT** need to the change the */etc/racoon.conf*, */etc/setkey.conf*, and */etc/psk.txt* files. Otherwise, modify these files according to your actual IP addresses.

The */etc/racoon.conf* file is given below.

```
path include "/etc";
path pre_shared_key "/etc/psk.txt";
path certificate "/etc/certs";
path script "/etc";

sainfo anonymous
{
    # pfs_group 2;
    lifetime time 1 hour ;
    encryption_algorithm 3des, blowfish 448, rijndael ;
    authentication_algorithm hmac_sha1, hmac_md5 ;
    compression_algorithm deflate ;
}

remote 172.17.0.2 # This should the IP of client
{
        exchange_mode aggressive, main;
        my_identifier address;
        mode_cfg on;
        script "p1_up_down" phase1_up;
        script "p1_up_down" phase1_down;
        proposal {
                encryption_algorithm 3des;
                hash_algorithm sha1;
                authentication_method pre_shared_key;
                dh_group 2;
```

```
        }
}
```

The */etc/setkey.conf* file is given below.

```
#!/usr/sbin/setkey -f
# Flush the SAD and SPD
flush;
spdflush;
spdadd 172.17.0.3 172.17.0.2 any -P out ipsec
        esp/transport//require;
spdadd 172.17.0.2 172.17.0.3 any -P in ipsec
        esp/transport//require;
# 172.17.0.3 is the IP of the server
# 172.17.0.2 is the IP of the client
```

The */etc/psk.txt* is given as follows.

```
172.17.0.2  realsecure
# realsecure is the secret key, you can use your key.
```

Start the IPsec service via the following commands.

```
setkey -f /etc/setkey.conf
racoon -Fd
```

You should see debug information as follows.

```
2019-02-02 03:28:44: INFO: Reading configuration from "/etc/racoon.conf"
2019-02-02 03:28:44: INFO: 127.0.0.0[500] used as isakmp port (fd=8)
...
2019-02-02 03:28:44: INFO: ::1[500] used as isakmp port (fd=14)
2019-02-02 03:28:44: INFO: ::1[4500] used as isakmp port (fd=15)
```

Moreover, the ping reply should have stopped.

2.3.1 Provide a screenshot of your "racoon -Fd" terminal results.

2.3.2 Provide a screenshot of your tcpdump terminal results.

2.3.3 Provide a screenshot of your ping terminal results.

2.4 Client Configuration

Open another terminal into your IPsec client, copy IPsec related configuration files to the */etc* folder.

```
docker exec -it client bash
chmod  600  /data/psk.txt
cp   /data/*   /etc/
```

The client configuration is similar to that of server. If your client IP address is 172.17.0.2 and server IP address is 172.17.0.3, you do **NOT** need to the change the *racoon.conf*, *setkey.conf*, and *psk.txt* files. Otherwise, modify these files according to your actual settings.

The */etc/racoon.conf* file.

```
path include "/etc";
path pre_shared_key "/etc/psk.txt";
path certificate "/etc/certs";
path script "/etc";

sainfo anonymous
{
    # pfs_group 2;
    lifetime time 1 hour ;
    encryption_algorithm 3des, blowfish 448, rijndael ;
    authentication_algorithm hmac_sha1, hmac_md5 ;
    compression_algorithm deflate ;
}

remote 172.17.0.3 # This should the IP of server
{
        exchange_mode aggressive, main;
        my_identifier address;
        mode_cfg on;
        script "p1_up_down" phase1_up;
        script "p1_up_down" phase1_down;
        proposal {
                encryption_algorithm 3des;
                hash_algorithm sha1;
                authentication_method pre_shared_key;
                dh_group 2;
        }
}
```

The */etc/setkey.conf* file.

```
#!/usr/sbin/setkey -f
```

```
# Flush the SAD and SPD
flush;
spdflush;
spdadd 172.17.0.2 172.17.0.3 any -P out ipsec
        esp/transport//require;
spdadd 172.17.0.3 172.17.0.2 any -P in ipsec
        esp/transport//require;
```

The */etc/psk.txt* file.

```
172.17.0.3   realsecure
# realsecure is the secret key, you can use your key.
```

Start the IPsec service via the following commands.

```
setkey -f /etc/setkey.conf
racoon -Fd
```

If successful, you should see IPsec Security Association (SA) information as follows.

```
INFO: IPsec-SA established: ESP/Transport 172.31.22.53[500]->54.92.159.233[500]
spi=253991427(0xf239a03)
```

The traffic monitor terminal would show the Security Parameter Index (SPI) of Encapsulating Security Payload (ESP).

```
IP 172.17.0.3 > 172.17.0.2: ESP(spi=0x0849b926,seq=0x1), length 100
```

2.4.1 Provide a screenshot of the IPsec connection information from the client side.

2.4.2 Provide a screenshot of tcpdump results.

2.4.3 Explain the concepts of ESP, SPI, SAD, and SPD.

3 Secure Shell (SSH) Port Forwarding

Port forwarding, or SSH tunneling, is a way to forward TCP traffic through a SSH connection. You can secure for example Telnet, FTP, POP3, SMTP and HTTP connections that would otherwise be insecure. There are two kinds of port forwarding: local and remote forwarding.

- Local port forwarding forwards traffic coming to a local port to a specified remote port. For example, you want to secure all traffic from a local client to a FTP server. You can set a SSH connection to forward traffic coming to port 2121 on the local client could be forwarded to port 21 (FTP) on the remote server. Local port forwarding is pretty common.
- Remote port forwarding does the opposite: it forwards traffic coming to a remote port to a specified local port. For example, all traffic coming to port 2323 on the server could be forwarded to port 23 (Telnet) on the client.

3.1 Lab Prep

Download the GitHub repository that contains the Docker configuration files for this lab.

```
git pull https://github.com/liranmatcu/Docker_Security_Labs.git
# If you have not downloaded the repository previously,
# you will need to use git clone instead of pull
```

Start the SSH and FTP service containers using *docker-compose*.

```
cd Docker_Security_Labs/ssh_port_forwarding

docker-compose up -d  # build and start containers
```

Two container instances will be built and started. One is named as ssh_server, and the other is named as ftp_server.

Obtain the IP addresses of the SSH and FTP service containers. Write them down for future use.

```
docker inspect ftp_server
docker inspect ssh_server
```

3.1.1 Provide a screenshot of your FTP server IP address.

3.1.2 Provide a screenshot of your SSH server IP address.

3.2 Test FTP Connections before Port Forwarding

Your local machine can function as a FTP client. Open a terminal, try connecting to your FTP server via the following commands. Note some of these commands may not receive any response.

```
nc -v FTP-Server-IP-address
# You can replace nc with ftp if the ftp command is available.
nc -v FTP-Server-IP-address 9000
nc -v FTP-Server-IP-address 9001
nc -v 127.0.0.1 9000
nc -v 127.0.0.1 9001
nc -v 127.0.0.1 9002
```

Windows users can download and install **nc (netcat)** from the following link: https://github.com/rsanchez-wsu/jfiles/wiki/Windows-10-Telnet-&-NetCat.

3.2.1 Were you able to establish a connection to the FTP server? Why?

3.3 Configure SSH Local Port Forwarding

Open another terminal on your local machine, and issue the following command to set the SSH port forwarding rule for FTP traffic.

```
ssh -L  9000:FTP-Server-IP-address:21  root@127.0.0.1  -p 9002
```

The password is **password**. If login succeeded, you are on the container FTP server! Do not close this terminal unless you want to terminate the SSH port forwarding.

3.3.1 What is the purpose/meaning of the above ssh command with -L?

3.4 Test FTP Connections after Port Forwarding

Open another terminal on your local machine, try to connect to the FTP server with the following commands. Again, note some of these commands may not receive any response.

```
nc -v FTP-Server-IP-address 9000
nc -v FTP-Server-IP-address 9001
nc -v 127.0.0.1 9000
nc -v 127.0.0.1 9001
```

3.4.1 Which command allows you to establish a connection to the FTP server? Why?

Hint: Check the docker-compose.yml file. To help you better answer/understand the above question, open a terminal into your FTP server, and start a tcpdump capture.

```
docker exec -it ftp_server bash

tcpdump -n tcp port 21 -vvv -i eth0 -w capture.txt
# or tcpdump -n tcp port 21 -vvv -w capture.txt
# FTP service uses TCP port 21
```

After starting the tcpdump capture, try one more FTP connection. Once you are done with the capture, view the captured traffic using the *strings* command.

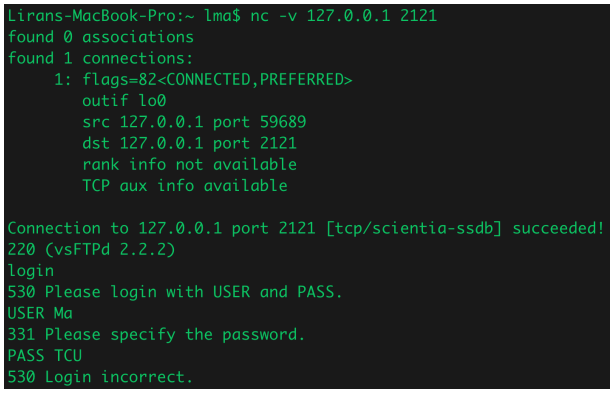
```
strings capture.txt
```

3.4.2 The "nc -v 127.0.0.1 9002" commands connects to what service? Why?

3.4.3 Provide a screenshot of a successful FTP connection using port forwarding.

3.4.4 Complete a FTP login session (use arbitrary user name and password) with and without SSH port forwarding. Check/Explain whether you can capture the user name and password in plaintext.

An example FTP login session using nc is shown in Fig. 3.



```
Lirans-MacBook-Pro:~ lma$ nc -v 127.0.0.1 2121
found 0 associations
found 1 connections:
     1: flags=82<CONNECTED,PREFERRED>
        outif lo0
        src 127.0.0.1 port 59689
        dst 127.0.0.1 port 2121
        rank info not available
        TCP aux info available

Connection to 127.0.0.1 port 2121 [tcp/scientia-ssdb] succeeded!
220 (vsFTPd 2.2.2)
login
530 Please login with USER and PASS.
USER Ma
331 Please specify the password.
PASS TCU
530 Login incorrect.
```

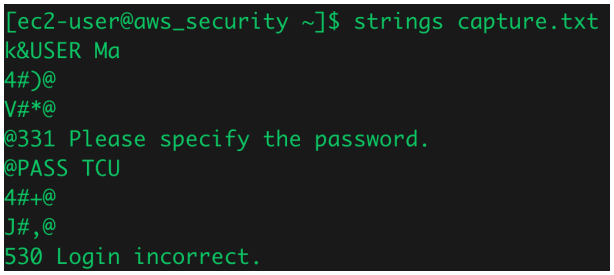
Figure 3: FTP login session.

An example tcpdump capture command targeting at port 22 (SSH) traffic is given below. You will need to run this command on the SSH server (not on the FTP server since it does not receive any data on port 22).

```
docker exec -it ssh_server bash

tcpdump -n tcp port 22 -vvv -w capture.txt
# or tcpdump -n tcp -vvv -i eth0 -w capture.txt
# SSH service uses port 22
```

An example of captured FTP login credentials is illustrated in Fig. 4.



```
[ec2-user@aws_security ~]$ strings capture.txt
k&USER Ma
4#)@
V#*@
@331 Please specify the password.
@PASS TCU
4#+@
]#,@
530 Login incorrect.
```

Figure 4: Captured FTP login credentials.

3.4.5 Assume that there is a web server whose IP address is 8.8.8.8, and it listens on port 8080. How would you modify the port forwarding command below to connect to the web server?

```
ssh -L  9000:FTP-Server-IP-address:21  root@127.0.0.1  -p 9002
```

3.4.6 (Optional) Design and implement a lab that utilizes remote port forwarding and dynamic port forwarding.

3.5 Prune Containers

Once you have completed this lab section, you may want to prune your containers to free system resources.

```
docker stop ssh_server ftp_server
docker rm ssh_server ftp_server
# You may use ``docker rmi'' to delete unwanted docker images.
```

4 General Questions

4.0.1 Any corrections or general comments about this lab?

4.0.2 How long did it take you to complete this lab?

4.0.3 Was it an appropriate length lab?